

Name: Vaishnavi Badoge

Aim: Implement and analyze various methods of studying correlation and regression including Karl Pearson's coefficient and simple linear regression.

Theory:

1. What is correlation? How is it different from regression?

Ans: Correlation is a statistical method used to measure the strength and direction of the relationship between two variables. It shows whether two variables increase or decrease together. The value of correlation ranges from -1 to +1, where +1 indicates perfect positive correlation, -1 indicates perfect negative correlation, and 0 indicates no relationship. Regression, on the other hand, is used to predict the value of one variable based on another variable. Correlation only measures association, while regression establishes a mathematical relationship and helps in prediction. For example, correlation can show that study hours and marks are related, while regression can predict marks based on study hours.

2. What is simple linear regression? Explain with suitable example.

Ans: Simple linear regression is a statistical technique used to study the relationship between one independent variable and one dependent variable. It predicts the value of the dependent variable using a straight-line equation. The general equation of simple linear regression is: $y = mx + c$ where y is the dependent variable, x is the independent variable, m is the slope, and c is the intercept. For example, if the number of hours studied affects exam marks, regression can be used to predict marks based on study hours. If students who study more generally score higher marks, the regression line helps estimate expected marks for a given number of study hours.

3. How do you calculate Pearson correlation in Python?

Ans: Pearson correlation in Python can be calculated using the `pearsonr()` function from the `scipy.stats` library. It measures the linear relationship between two variables. The value ranges from -1 to +1. A value close to +1 indicates strong positive correlation, a value close to -1 indicates strong negative correlation, and a value near 0 indicates weak or no correlation.

4. What is multicollinearity? Does it affect simple regression?

Ans: Multicollinearity occurs when two or more independent variables in a regression model are highly correlated with each other. This creates difficulty in understanding the individual effect of each variable on the dependent variable. It mainly affects multiple regression models because they contain more than one independent variable. Simple linear regression uses only one independent variable, so multicollinearity does not affect it. However, in multiple regression, multicollinearity can reduce prediction accuracy and make regression coefficients unstable.

5. A dataset shows strong correlation but poor prediction accuracy. Why?

Ans: A dataset may show strong correlation but still have poor prediction accuracy because correlation only measures the strength of the relationship and not the quality of prediction. Prediction accuracy can become poor due to factors such as outliers, insufficient data, non-linear relationships, overfitting, missing variables, or noise in the data. For example, two variables may move together strongly, but if the relationship is not linear, a simple regression model may fail to predict correctly. Therefore, high correlation does not always guarantee accurate predictions.

```
import pandas as pd

# Sample data
data = {
    'x': [10, 20, 30, 40, 50],
    'y': [15, 25, 35, 45, 55]
}
df = pd.DataFrame(data)

# Calculate correlation between two columns
corr = df['x'].corr(df['y'])

print(f"Pearson Correlation Coefficient: {corr}")
```

Pearson Correlation Coefficient: 1.0

```
from sklearn.linear_model import LinearRegression
import numpy as np

# 1. Independent variable (Years of Experience) - reshaped to 2D array
x = np.array([1, 2, 3, 5, 8]).reshape(-1, 1)

# 2. Dependent variable (Salary in $1000s)
y = np.array([45, 50, 60, 80, 110])

# 3. Create and train the model
model = LinearRegression()
model.fit(x, y)
```

```
# 4. Predict salary for 10 years of experience
new_input = np.array([[10]])
prediction = model.predict(new_input)

# 5. Output results
print(f"Predicted Salary for 10 years: ${prediction[0]:.2f}k")
print(f"Regression Slope: {model.coef_[0]:.2f}")
print(f"Regression Intercept: {model.intercept_[0]:.2f}")
```

```
Predicted Salary for 10 years: $128.18k
Regression Slope: 9.55
Regression Intercept: 32.73
```

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 1. Prepare the data
# x must be a 2D array (column vector), so we use .reshape(-1, 1)
x = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 6, 8, 10])

# 2. Create and train the Linear Regression model
model = LinearRegression()
model.fit(x, y)

# 3. Display the results
# model.coef_ returns an array, so we take the first element [0]
slope = model.coef_[0]
intercept = model.intercept_[0]

print(f"Slope (m): {slope}")
print(f"Intercept (c): {intercept}")
print(f"Linear Equation: y = {slope}x + {intercept}")
```

```
Slope (m): 2.0
Intercept (c): 0.0
Linear Equation: y = 2.0x + 0.0
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Step 1: Updated Sample Data
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Marks_Obtained': [25, 39, 47, 54, 64, 65, 69, 78, 80, 84]
}
df = pd.DataFrame(data)

# Step 2: Split data into input (X) and output (y)
X = df[['Hours_Studied']] # Independent variable
y = df['Marks_Obtained'] # Dependent variable

# Step 3: Train the Linear Regression Model
model = LinearRegression()
model.fit(X, y)

# Step 4: Get model parameters
slope = model.coef_[0]
intercept = model.intercept_[0]
print(f"Regression Equation: Marks = {slope:.2f} * Hours + {intercept:.2f}")

# Step 5: Predict values
y_pred = model.predict(X)

# Step 6: Evaluate the Model
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R² Score: {r2:.4f}")

# Step 7: Visualize the regression line
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Actual Data', s=100)
plt.plot(X, y_pred, color='red', linewidth=2, label='Regression Line')
plt.title('Simple Linear Regression: Hours Studied vs Marks Obtained')
plt.xlabel('Hours Studied')
plt.ylabel('Marks Obtained')
```

```
plt.grid(True)  
plt.legend()  
plt.show()
```

Regression Equation: Marks = 6.18 * Hours + 26.53
Mean Squared Error (MSE): 14.40
R² Score: 0.9563

